

BABU BANARASI DAS UNIVERSITY

ACADEMIC YEAR 2026-2027



SCHOOL OF COMPUTER APPLICATIONS

NoSQL Project On

STUDENT MANAGEMENT SYSTEM USING MONGODB

Submitted to:

Mr. Harendra Singh

Department: BCA (DS & AI)

Date: 25th September 2026

Submitted by:

Sanskriti Puri

Program: BCA (DS & AI)

Roll No: [University Roll No]

ACKNOWLEDGEMENT

I sincerely express my gratitude to my teacher **Mr. Harendra Singh** for giving me the opportunity to work on this NoSQL project titled 'Student Management System Using MongoDB'.

This project helped me understand the core concepts of NoSQL databases and learn how MongoDB is used to install, configure, create, retrieve, update, sort, and delete data. I also learned how different query operators such as equal to (\$eq), greater than (\$gt), less than (\$lt), greater than or equal to (\$gte), less than or equal to (\$lte), logical AND (\$and), and logical OR (\$or) work in MongoDB.

I am deeply thankful to my teacher for his continuous guidance, technical encouragement, and support provided throughout the completion of this project.

Sanskriti Puri

BCA (DS & AI)

School of Computer Applications

Babu Banarasi Das University, Lucknow

INTRODUCTION

MongoDB is a popular NoSQL database management system that stores data in the form of flexible, JSON-like documents called BSON (Binary JSON). Unlike traditional relational databases, MongoDB does not require data to be stored in rigid tables with fixed rows and columns.

In this project, a comprehensive Student Management System has been created using MongoDB Community Server 8.3 and MongoDB Compass. The database contains verified records for **65 students** (satisfying the strict project mandate of having at least 60 data records). Each student document stores essential academic and demographic details including Roll Number, Student Name, Age, Examination Marks, and City of residence across Uttar Pradesh (Lucknow, Kanpur, Ayodhya, Varanasi, Prayagraj, Gorakhpur, Jaunpur, Bareilly, and Gonda).

Different MongoDB operations have been executed on the student dataset: single document insertion (insertOne), bulk dataset ingestion (insertMany), displaying records, updating documents, deleting single and multiple records, sorting in ascending and descending order, counting documents, and searching by attribute.

OBJECTIVES

- To understand the core architecture of NoSQL document databases.
- To install, configure, and manage MongoDB Community Edition Server on Windows.
- To connect and manage databases using MongoDB Compass GUI and MongoDB Shell (mongosh).
- To create a dedicated database ('Students') and collection ('students').
- To insert at least 60 student records (65 records implemented using insertOne and insertMany).
- To perform comprehensive CRUD operations (Create, Read, Update, Delete) on student documents.
- To demonstrate comparison query operators: \$eq, \$gt, \$lt, \$gte, and \$lte.
- To demonstrate compound logical operators: \$and and \$or.
- To perform sorting operations in ascending (1) and descending (-1) order.
- To utilize pagination, limit methods, and document counts.

TOPIC 1: INSTALLATION & CONFIGURATION OF MONGODB

1. Installing MongoDB Community Edition: Downloaded the Windows x64 MSI package from [mongodb.com](https://www.mongodb.com). Selected Complete installation and enabled 'Install as Windows Service'. This registers the service on port 27017.

```
# Managing MongoDB Service in Windows PowerShell:  
Get-Service -Name MongoDB # Checks status: Running  
net start MongoDB # Starts MongoDB Server  
net stop MongoDB # Stops MongoDB Server
```

2. Connecting via MongoDB Compass GUI: Launched MongoDB Compass, entered `mongodb://localhost:27017`, and established a visual connection. The dashboard allows visual exploration of databases and provides the embedded `_MONGOSH` terminal.

3. Connecting via MongoDB Shell (mongosh): Opened terminal and connected using `mongosh` "`mongodb://localhost:27017`" to run commands interactively.

TOPIC 2: WORKING WITH DOCUMENTS & COLLECTIONS

1. CREATE DATABASE

```
use Students
```

Explanation: Creates or switches to the Students database.

```
switched to db Students
```

2. CREATE COLLECTION

```
db.createCollection("students")
```

Explanation: Creates a collection named students.

```
{ ok: 1 }
```

3. INSERT SINGLE STUDENT (insertOne)

```
db.students.insertOne({ roll: 1, name: "Anamika", age: 19, marks: 88, city: "Lucknow" })
```

Explanation: Demonstrates single document insertion using `insertOne()`.

```
{ acknowledged: true, insertedId: ObjectId("6ab2bc7e633f176f018cde37") }
```

4. INSERT MULTIPLE STUDENTS (insertMany - 64 Records, Total 65)

```
db.students.insertMany([
  {roll:2, name:"Uday", age:18, marks:76, city:"Jaunpur"},
  {roll:3, name:"Vaishnavi", age:19, marks:92, city:"Kanpur"},
  ...
  {roll:65, name:"Sanskriti", age:19, marks:97, city:"Lucknow"}
])
```

Explanation: Ingests 64 student documents into 'students'. The collection now contains exactly 65 records, fulfilling the >= 60 data mandate.

```
{ acknowledged: true, insertedIds: { '0': ObjectId(...), ... '63': ObjectId(...) } }
// db.students.countDocuments() => 65
```

5. DISPLAY ALL STUDENTS

```
db.students.find()
```

Explanation: Retrieves all documents from the students collection.

```
[ { roll: 1, name: 'Anamika', age: 19, marks: 88, city: 'Lucknow' }, ... ] (65 docs)
```

6. EQUAL TO OPERATOR – \$eq

```
db.students.find({ marks: { $eq: 88 } })
```

Explanation: Finds students whose marks are exactly 88.

```
[ { roll: 1, name: 'Anamika', marks: 88 }, { roll: 27, name: 'Ananya', marks: 88 }, { roll: 53, name: 'Tushar', marks: 88 } ]
```

7. GREATER THAN OPERATOR – \$gt

```
db.students.find({ marks: { $gt: 85 } })
```

Explanation: Finds students scoring greater than 85 marks (23 students found).

```
[ { roll: 1, name: 'Anamika', marks: 88 }, { roll: 3, name: 'Vaishnavi', marks: 92 }, { roll: 65, name: 'Sanskriti', marks: 97 }, ... ]
```

8. LESS THAN OPERATOR – \$lt

```
db.students.find({ marks: { $lt: 60 } })
```

Explanation: Finds students whose marks are less than 60.

```
[ { roll: 18, name: 'Jai', marks: 58 }, { roll: 33, name: 'Jai', marks: 55 }, { roll: 47, name: 'Aman', marks: 59 } ]
```

9. GREATER THAN OR EQUAL TO OPERATOR – \$gte

```
db.students.find({ marks: { $gte: 90 } })
```

Explanation: Finds all students scoring 90 or above.

```
[ { roll: 3, name: 'Vaishnavi', marks: 92 }, { roll: 12, name: 'Shreya', marks: 95 }, { roll: 65, name: 'Sanskriti', marks: 97 }, ... ]
```

10. LESS THAN OR EQUAL TO OPERATOR – \$lte

```
db.students.find({ marks: { $lte: 70 } })
```

Explanation: Finds all students scoring 70 or below.

```
[ { roll: 4, name: 'Annu', marks: 69 }, { roll: 10, name: 'Vivek', marks: 65 }, { roll: 20, name: 'Omkar', marks: 63 }, ... ]
```

11. UPDATE OPERATION (updateOne)

```
db.students.updateOne({ roll: 5 }, { $set: { marks: 95 } })
```

Explanation: Updates the marks of student Roll 5 (Vansh) to 95.

```
{ acknowledged: true, matchedCount: 1, modifiedCount: 1 }
```

12. UPDATE MANY OPERATION (updateMany)

```
db.students.updateMany({ marks: { $gte: 90 } }, { $set: { grade: "A+" } })
```

Explanation: Assigns grade: 'A+' to all students scoring >= 90.

```
{ acknowledged: true, matchedCount: 15, modifiedCount: 15 }
```

13. DELETE ONE OPERATION (deleteOne)

```
db.students.deleteOne({ roll: 10 })
```

Explanation: Deletes one student document matching roll number 10 (Vivek).

```
{ acknowledged: true, deletedCount: 1 }
```

14. DELETE MANY OPERATION (deleteMany)

```
db.students.deleteMany({ marks: { $lt: 60 } })
```

Explanation: Removes all students with marks below 60.

```
{ acknowledged: true, deletedCount: 3 } // Removed Roll 18, 33, 47
```

15. AND OPERATION – \$and

```
db.students.find({ $and: [ { marks: { $gt: 80 } }, { age: 19 } ] })
```

Explanation: Finds students satisfying both conditions: marks > 80 and age = 19.

```
[ { roll: 1, name: 'Anamika', marks: 88 }, { roll: 3, name: 'Vaishnavi', marks: 92 }, { roll: 65, name: 'Sanskriti', marks: 97 } ]
```

16. OR OPERATION – \$or

```
db.students.find({ $or: [ { city: "Lucknow" }, { city: "Kanpur" } ] })
```

Explanation: Finds students residing in either Lucknow or Kanpur.

```
[ { roll: 1, name: 'Anamika', city: 'Lucknow' }, { roll: 3, name: 'Vaishnavi', city: 'Kanpur' }, ... ]
```

17. SORT OPERATION (Ascending & Descending)

A. Ascending Order: db.students.find().sort({ marks: 1 }).limit(3)

```
[ { roll: 28, name: 'Bhaskar', marks: 61 }, { roll: 43, name: 'Sarika', marks: 62 }, { roll: 20, name: 'Omkar', marks: 63 } ]
```

B. Descending Order: db.students.find().sort({ marks: -1 }).limit(3)

```
[ { roll: 65, name: 'Sanskriti', marks: 97 }, { roll: 35, name: 'Krishna', marks: 96 }, { roll: 5, name: 'Vansh', marks: 95 } ]
```

18. LIMIT OPERATION

```
db.students.find().limit(5)
```

Explanation: Restricts output to the first 5 records.

```
[ { roll: 1, name: 'Anamika' }, { roll: 2, name: 'Uday' }, { roll: 3, name: 'Vaishnavi' }, { roll: 4, name: 'Annu' }, { roll: 5, name: 'Vansh' } ]
```

19. SEARCH BY CITY

```
db.students.find({ city: "Lucknow" })
```

Explanation: Finds all students located in Lucknow (20 students found).

```
[ { roll: 1, name: 'Anamika', city: 'Lucknow' }, { roll: 8, name: 'Pratik', city: 'Lucknow' }, { roll: 65, name: 'Sanskriti', city: 'Lucknow' } ]
```

20. COUNT DOCUMENTS

```
db.students.countDocuments()
```

Explanation: Counts total active records remaining after deletions (65 initial - 1 deleteOne - 3 deleteMany = 61).

```
61
```

MONGODB OPERATORS SUMMARY REFERENCE

Operator	Type	Meaning	Example Query
\$eq	Comparison	Matches values equal to specified value	db.students.find({marks: {\$eq: 88}})
\$gt	Comparison	Matches values greater than specified value	db.students.find({marks: {\$gt: 85}})
\$lt	Comparison	Matches values less than specified value	db.students.find({marks: {\$lt: 60}})
\$gte	Comparison	Matches values greater than or equal	db.students.find({marks: {\$gte: 90}})
\$lte	Comparison	Matches values less than or equal	db.students.find({marks: {\$lte: 70}})
\$and	Logical	Joins query clauses with logical AND	db.students.find({\$and: [{marks:{\$gt:80}}, {age:19}]})
\$or	Logical	Joins query clauses with logical OR	db.students.find({\$or: [{city:'Lucknow'}, {city:'Kanpur'}]})
\$set	Update	Modifies or assigns field value	db.students.updateOne({roll:5}, {\$set:{marks:95}})

CITY-WISE STUDENT PERFORMANCE ANALYTICS

City Name	Students	Average Marks	Top Scorer Achieved
Lucknow	20	87.85%	97 (Sanskriti)
Varanasi	7	85.57%	95 (Vansh)
Kanpur	9	81.11%	92 (Vaishnavi)
Prayagraj	7	80.14%	96 (Krishna)
Ayodhya	5	76.20%	86 (Divya)
Jaunpur	6	73.83%	86 (Saumya)
Gorakhpur	4	69.00%	78 (Sangita)
Bareilly	3	68.00%	77 (Mohit)

CONCLUSION

This project provided in-depth practical knowledge of NoSQL databases using MongoDB. A Student Management System containing 65 student records across Uttar Pradesh was designed and managed. Through this project, I gained practical hands-on experience in:

1. Installing and configuring MongoDB Community Server and connecting via Compass and mongosh.
2. Inserting single (insertOne) and bulk (insertMany) student documents.
3. Retrieving data using advanced comparison operators (\$eq, \$gt, \$lt, \$gte, \$lte) and logical operators (\$and, \$or).
4. Updating records with atomic operators (\$set) and performing targeted single and bulk deletions.
5. Organizing data through sorting (ascending and descending), pagination limits, and attribute searches.

Overall, this project provided valuable practical understanding of how MongoDB enables modern academic institutions to manage, query, and analyze student records with high efficiency and schema flexibility.